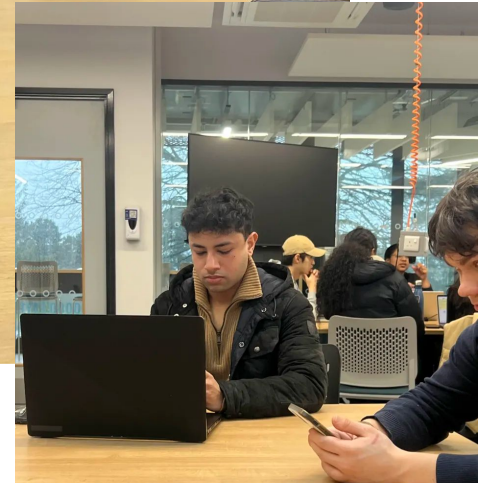


AI Failures Museum

Prototype v0.1.0 – Vertical Slice

Team HL



The Problem:

AI failures can be detrimental as they often output incorrect information but present it as accurate, which leads users to accept results at face value. This is very dangerous because these flawed findings can guide decision-making in high-stakes cases where accuracy is critical.

Real-world deployments often fail due to hidden biases or false precision.

Our Solution:

A platform to document these failures.

Our goal is not to blame, but to facilitate learning and responsible practice.

We aim to accomplish this by displaying artefacts of real world examples, and then providing an opportunity to test the reader's knowledge and gamifying this.

Today's Demo:

A "Vertical Slice" showcasing one complete user journey: *The Rivertown Flood Map Failure*.

Exhibit 01: The "False Sense of Security"

The Decision Context:

Welcome to the exhibit on the Rivertown Flood Map. This is about urban planning in 2019.

The Rivertown City Council intended to make their zoning process more up-to-date for a new housing estate. They used their AI system to replace slow, manual surveying with a rapid, automated tool that helps people make decisions.

The goal was to be efficient by automatically putting land into one of two categories: 'Zone A: Safe' or 'Zone B: High Risk'.

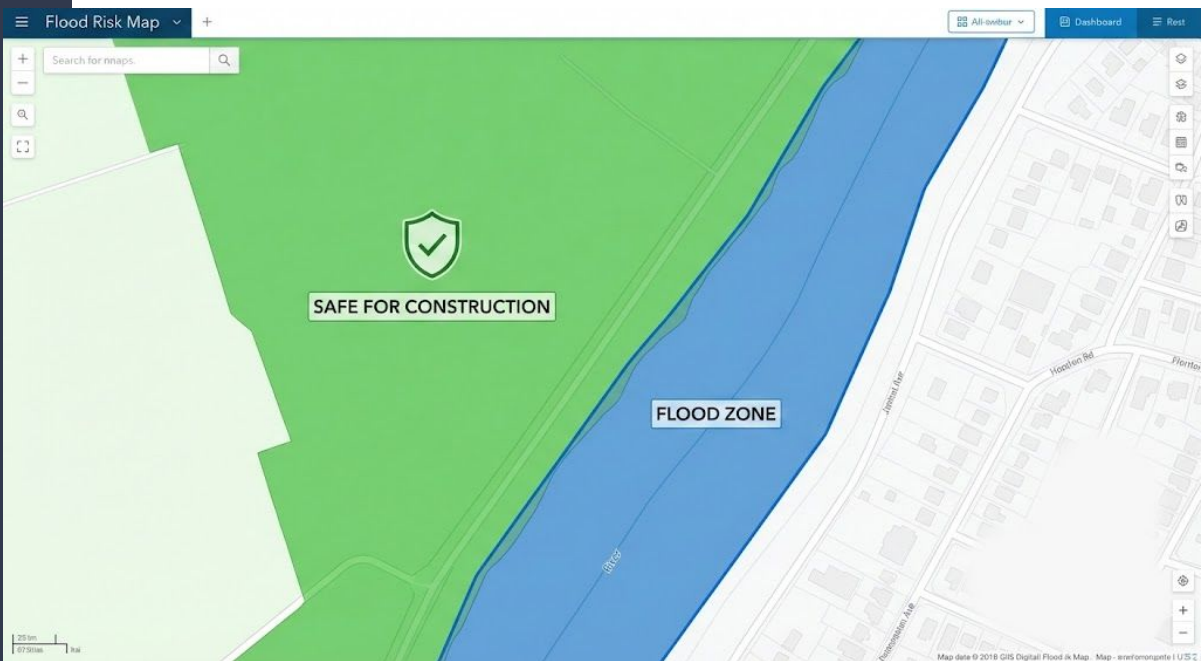


Exhibit 01: The "False Sense of Security"

What the AI System Claimed:

The system said it was quite accurate. It employed LiDAR scans with high resolution and 50 years of rainfall data from the past.

The system created a sharp, binary blue line, as you can see in this picture. Everything behind it was marked green, which meant "0% Risk."

"It gave the impression of perfect certainty if you were one metre behind the queue, the AI said you were safe."

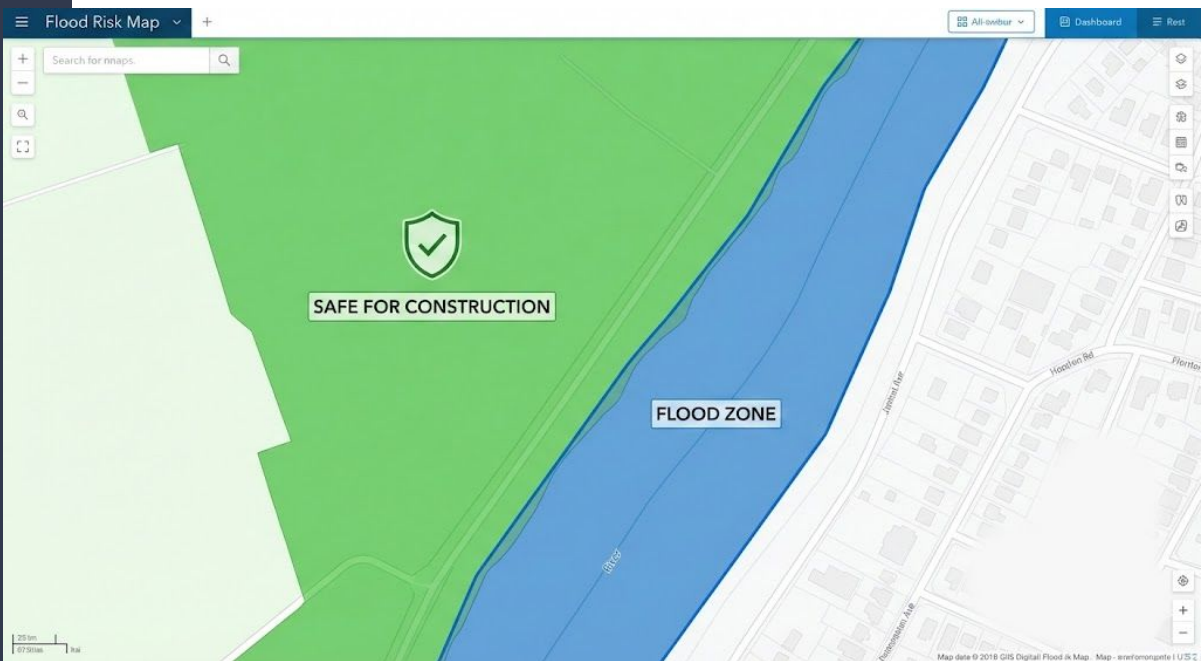


Exhibit 01: The "False Sense of Security"

What Went Wrong & Contributing Factors:

In 2021, a 1-in-100-year storm hit. The rainfall was 15% higher than anything in the training data.

The failure was the False Sense of Security. 500 families were flooded, and because the AI had told them they were '0% Risk,' 90% of them had no flood insurance.

Our analysis highlights two key contributing factors:

1. **The Stationarity Assumption:** The model assumed the future climate would look like the past 50 years, ignoring climate change.
2. **False Precision:** The dashboard hid the uncertainty. It gave a binary 'Yes/No' answer instead of a risk probability, leading to Automation Bias where officials stopped questioning the output.

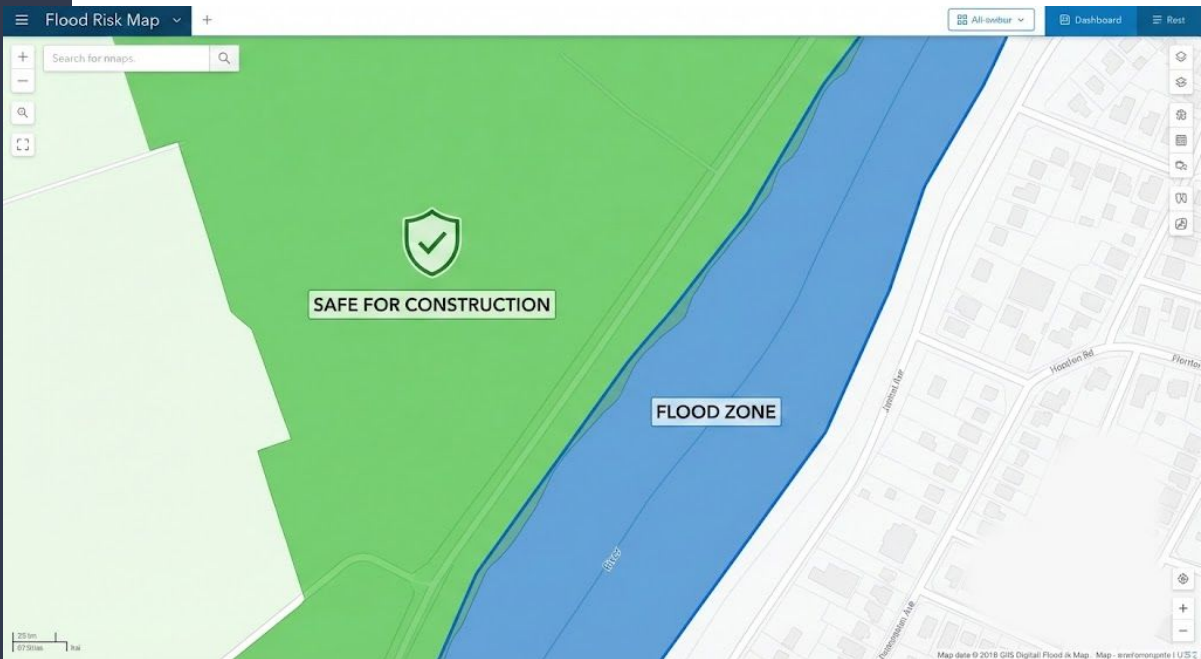


Exhibit 01: The "False Sense of Security"

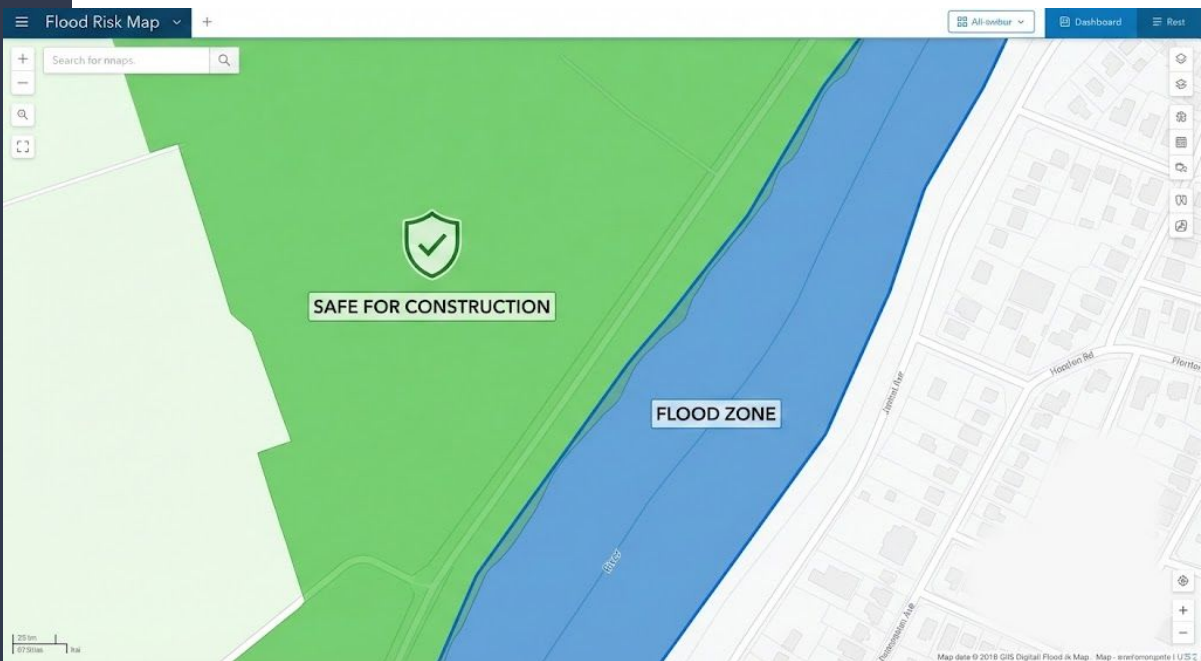
Lessons Learned:

This exhibit teaches us three important things that developers in the future should know:

Data Limitation: AI can't guess what will happen in "Black Swan" situations if they aren't in the training data from the past.

We need to be able to see uncertainty in our design choices. Instead of a hard binary line, a gradient map that faded from red to green would have advised consumers of the risk.

Governance: "Human-in-the-loop" supervision is necessary for edge cases close to flood walls.



Demo



We built the museum using a standard Django architecture to ensure maintainability. We prioritized professional practice by setting up a full documentation hub and automated testing from Day 1

Where key documentation is located:

We put all of the documents under this **docs/** subdirectory so that the root stays clean. This encompasses our choices about architecture and our moral concerns.

How to run the system locally:

To run locally, a developer simply clones the repo and runs

```
pip install -r requirements.txt
```

We've documented the specific startup commands in the README.

How tests are executed:

We have a suite of automated unit tests located in this **tests/** folder. They can be executed via the standard Django command **python manage.py test**, which runs our model and view verifications.

Key issues or milestones tracked:

We used GitHub Issues to track our work. As you can see here, we broke down the 'Earthquake Exhibit' requirement into smaller tasks and moved them to 'Done' only when the tests passed.

AI-Failure-Museum/	
├─ README.md	← START HERE: Project overview & quick start
├─ docs/	← DOCUMENTATION HUB
│ ├── SETUP.md	→ Detailed installation guide
│ ├── TESTING.md	→ How to run tests, coverage reports
│ └── ARCHITECTURE.md	→ System design & technical
├─ decisions	
│ ├── USER_GUIDE.md	→ End-user documentation
│ └── DEVELOPMENT.md	→ Developer guidelines
├─ ai_museum/	← DJANGO PROJECT ROOT
│ ├── settings.py	→ Project configuration
│ ├── urls.py	→ URL routing
│ ├── wsgi.py	→ WSGI application
│ └── asgi.py	→ ASGI application (if async)
├─ exhibits/	← MAIN APPLICATION (Core Features)
│ ├── models/	
│ │ ├── exhibit.py	→ Exhibit model (AI failure cases)
│ │ ├── quiz.py	→ Quiz & question models
│ │ └── user_profile.py	→ User extensions
│ ├── views/	
│ │ ├── exhibit_views.py	→ Browse, detail, search
│ │ ├── curator_views.py	→ Curator dashboard & tools
│ │ └── quiz_views.py	→ Quiz functionality
│ ├── templates/	
│ │ └── exhibits/	→ HTML templates
│ ├── tests/	← TEST SUITE
│ │ ├── test_models.py	→ Model unit tests
│ │ ├── test_views.py	→ View integration tests
│ │ ├── test_quiz.py	→ Quiz logic tests
│ │ └── test_permissions.py	→ Access control tests
│ ├── urls.py	→ App-level URL patterns
│ ├── forms.py	→ Django forms
│ └── admin.py	→ Admin interface config
├─ static/	← STATIC ASSETS
│ ├── css/	
│ │ └── styles.css	→ Custom styles
│ ├── js/	
│ │ └── main.js	→ JavaScript functionality
│ └── images/	→ Images, logos
├─ media/	← USER UPLOADS (gitignored)
│ └── exhibit_images/	
├─ .github/	← GITHUB WORKFLOWS
│ ├── workflows/	
│ │ ├── tests.yml	→ CI: Run tests on push
│ │ └── deploy.yml	→ CD: Deploy to production
├─ requirements.txt	← PYTHON DEPENDENCIES
├─ .gitignore	← Git exclusions
├─ manage.py	← Django management commands
├─ db.sqlite3	← Database (gitignored)
└─ LICENSE	← Project license

Sprint 2 Roadmap (The Path to v1.0)

Finally, here is our strategy for Sprint 2 to get to the final v1.0 release.

First, let's talk about Features: we're going from this small part of the museum to the whole thing. We will add the 20 exhibits we need from different categories to our dataset. Most importantly, we will finish the Curator Dashboard, which will let moderators fully edit exhibitions and submit supporting items like maps and images directly through the UI.

Second, about accessibility: we want this technology to be available to everyone. We will do rigorous inspections to make sure that our colour contrast and screen-reader compatibility fulfil the specifications for the module.

Finally, Managing Risks: Our top priorities are safety and stability. To protect our database, we are introducing a full set of automated tests and making SQL checks more severe. This makes sure that we don't mistakenly mess up the main visitor experience when we implement these new features.